

项目信息表

项目名称	MusicQuery Agent——基于自然语言交互的音乐作品库智能查询系统				
团队名称	旋律引擎团队				
团队成员		姓名	班级	电话	学号
	队长	【待填写】	【待填写】	【待填写】	【待填写】
	队员	【待填写】	【待填写】	【待填写】	【待填写】
	队员	—	—	—	—
指导教师	【待填写】				
项目的背景和问题	（一）行业背景 音乐产业正在经历深度数字化：从词曲版权管理、演出票务、粉丝运营到播放数据统计，各类音乐机构的核心资产越来越多地沉淀在数据库中。公开资料显示，我国现存音乐娱乐相关企业数十万家，其				

	中绝大多数是独立音乐人工作室、中小型厂牌、Livehouse、音乐培训机构和校园音乐社团——它们普遍缺乏专职数据工程师，数据分析需求却真实存在：哪首歌播放量最高？哪个城市的粉丝增长最快？最近一个月哪个渠道带来的注册用户最多？ （二）问题定义 上述机构面临一个共同的“数据看得见却够不着”的困境，具体表现为三点：第一，查询依赖 SQL 技能。业务数据通常存放在 MySQL 等关系型数据库中，而 SQL 对非技术人员有较高学习门槛，机构往往需要临时委托技术人员写查询，一次小改动就要来回沟通。第二，通用 BI 工具不适配。市面主流 BI 产品面向中大型企业，价格高、部署重、配置复杂，中小机构“用不起也用不会”。第三，人工智能工具直接使用有风险。通用大语言模型虽能生成
--	--

	SQL，但直接对接生产数据库存在误删数据、SQL 注入、结果不可解释等安全隐患，缺乏针对数据库场景的防护与容错机制。 （三）问题的价值 让“不懂 SQL 的人”也能安全地问数据库要答案，本质上是打通数据价值变现的最后一公里。谁能把自然语言交互、数据库安全和企业级容错以低成本的方式组合起来，谁就能服务这个长期被忽视的中小机构市场。这正是本项目要解决的核心问题。 （四）需求紧迫性小结 综合来看，目标客户群的需求呈现“高频、低门槛、强安全”三个特征：数据查询是每周都会发生的重复劳动；使用者不愿学习 SQL；数据资产不容被破坏。三个特征共同指向一个结论——市场需要的不是又一个“会写 SQL 的聊天机器人”，而是一套面向
--	--

	数据库场景、把安全与容错做进架构的完整产品。 这一结论经小组两轮讨论确认，成为后续全部设计决策的出发点。 （四）需求调研方法 为验证上述问题是否真实存在，团队开展了两类调研：一是实地访谈——在成员所在校园音乐社团访谈 5 名社团干部与 2 名指导老师，归纳出高频数据查询场景前五位：播放量排名、用户城市分布、时段活跃度、歌曲时长分布、渠道注册转化——这五个场景在本项目中均可查询验证；二是案例整理——收集 3 位独立音乐人公开访谈中提及的数据困扰，与访谈结论相互印证。调研结论：目标客户普遍表示“数据有用但不会取”，若工具免费易用，愿意立刻上手。
解决方案	（一）总体方案

	针对上述问题，我们提出并实现了 MusicQuery Agent——基于自然语言交互的音乐作品库智能查询系统。系统以“数据库设计是核心、Agent 是交互层”为架构原则，将一次用户提问拆解为四步流水线：第一步意图判断，由大模型充当守门员判断问题是否与音乐业务相关，无关问题直接礼貌拦截；第二步 SQL 生成，把数据库建表语句（DDL）与字段业务说明注入提示词，让大模型严格依据真实表结构生成只读查询；第三步执行校验，通过语句白名单、危险关键词黑名单、全程日志留痕三道安全防线后执行 SQL，若执行报错则把报错信息回传给大模型自动修正，最多重试两次；第四步结果解释，将查询结果转化为不超过 50 字的中文结论返回用户。 （二）方案的产出过程
--	---

	本方案并非一次成型。选题阶段小组召开了两次专题讨论：第一次聚焦“做什么”，对比了音乐社交、版权管理、数据问答三个方向，基于“技术可行性高、课程契合度强、演示效果直观”三条标准一致选定数据问答；第二次聚焦“怎么做”，成员分别调研了直接拼接生成与分步流水线两种路线，讨论认为分步流水线在出错定位、安全管控上明显占优，由此确定四步架构。开发阶段采用“AI 辅助编码 + 人工验收”模式：AI 负责生成初版代码与文案，团队成员负责架构决策、逐行验收和改动实验，所有关键代码均通过 Git 版本管理并在 GitHub 开源。 （三）关键技术决策 讨论中还敲定了三项关键决策：一是提示词中的表结构在运行时直接读取建表脚本，实现“单一数据源”，避免提示词与真实表结构脱节；二是设置温度
--	--

	参数为 0 保证生成确定性；三是建立报错回传机制，将 MySQL 报错原文作为修复上下文，这是系统可靠性的关键一环。 （四）方案与课程知识的对应 作为数据库课程项目，本方案将课程核心知识点逐一落地：建库建表（DDL）、主键自增、外键约束、ENUM 域约束对应完整性设计；单列与复合索引、最左前缀原则、覆盖索引对应物理设计与性能优化，并已用 EXPLAIN 执行计划逐一实证；事实表与维度表的划分对应数据建模思想；参数化查询对应 SQL 注入防护；事务的提交与回滚保证批量数据导入的原子性。可以说，课程大纲中的关键概念在本项目中均有真实落点，且可用可复现的实验证据支撑。 （四）流水线各步骤的输入与输出 为使方案可工程化落地，小组明确了每一步的输入
--	--

	输出契约：意图判断以用户原文为输入、以 YES/NO 为输出；SQL 生成以原文加 DDL 为输入、输出单条 SELECT 语句；执行校验以 SQL 为输入，输出结果集或失败原因（含自修复循环）；结果解释以结果集 JSON 为输入，输出 50 字结论。任一环节失败都有明确定义的降级路径：大模型不可用时界面给出重试引导；解释生成失败时退化为“共 N 条记录”的固定话术——保证核心查询链路永远可用。
产品、服务	（一）产品形态 本项目提供的产品是一个可直接部署的 Web 应用（B/S 架构）与配套的开源代码仓库。用户打开网页，在输入框用中文提问，页面下方同步呈现三块内容：系统生成的 SQL（供技术人员复核）、查询结果表格（含条数与执行耗时）、以及一段自然语言结论（供业务人员阅读）；侧边栏提供本次耗

	时、成功状态与最近十条历史查询记录，所有请求均可追溯。 （二）功能清单 1. 自然语言数据问答：覆盖统计（COUNT/SUM/AVG）、多表 JOIN、排序 TOP-N、条件过滤、时间范围等高频查询模式；2. 意图守门：非音乐业务问题自动识别并礼貌拒绝；3. 安全防护：SELECT 白名单、危险关键词正则黑名单、参数化查询防注入、被拦截请求留痕；4. 失败自修复：SQL 执行报错自动回传修正，最多重试两次，修正过程可见；5. 查询日志：问题、生成的 SQL、是否成功、耗时全部入库；6. 演示数据生成器：内置按任务书规模（50 歌手/100 专辑/500 歌曲/200 用户/5000 播放记录）生成可复现的仿真数据。 （三）服务模式
--	---

面向不同客户提供三层服务：开源版（代码公开，自行部署）、托管版（我们代部署与运维的订阅服务）、定制版（对接客户已有数据库与业务系统的一次性开发）。此外提供完整的技术文档与答辩级演示材料，降低客户试用门槛。

（三）界面与开源生态

主界面按“三块式”设计：生成的 SQL 以代码块呈现，供技术人员复核；结果表格支持滚动浏览，供数据人员核对；50 字结论高亮展示，供业务人员直读。侧边栏历史记录可展开查看当时的 SQL 与耗时，形成“查询—留痕—复盘”闭环。项目在 GitHub 公开并配套完整文档：README 五步部署指南、schema.sql 建库脚本、import_data.py 数据生成器、答辩演示文稿及其生成脚本，全部经全历史密钥扫描后发布。

客户和用户画像	（一）客户画像（出钱的人） 客户一：独立厂牌/音乐人工作室主理人。25-40岁，管理着几百到几千首作品与粉丝数据，懂业务但不懂 SQL，预算敏感，希望“花小钱把数据用起来”。痛点：每次看数据都要找技术朋友帮忙。 客户二：音乐培训机构运营负责人。关心学员增长、续费率、课程相关的曲目使用统计，采购决策周期短，看重“开箱即用”。 客户三：高校音乐社团指导老师/学生干部。预算极低但需求高频（活动复盘、成员活跃度），是产品的种子用户与口碑传播节点。 （二）用户画像（使用的人） 用户一：经纪人助理/运营专员，22-30 岁，每天多次查看数据，期望“像聊天一样问数据”，对响应速度和答案准确性敏感。
----------------	---

	用户二：培训教师/社团管理员，30-45 岁，每周一至两次做例会汇报，重视结果的准确与可解释（因此产品同时给出 SQL 供复核）。 （三）画像小结 客户与用户的共同特征是：数据有价值、SQL 是鸿沟、预算有限、要求安全可靠。产品“自然语言问、SQL 透明可查、双输出”的设计正是针对这一组合特征。该画像经小组讨论并结合团队成员在校内音乐社团的实地访谈形成。 （三）典型使用场景 场景一：周一早上，社团管理员小林打开系统问“上周新增的用户来自哪些城市？”，几秒钟得到答案，直接贴进例会纪要——旧流程中这需要找技术部排队半天。场景二：厂牌主理人在发新专辑前问“上一张专辑播放量最高的三首歌的听众分布在哪些城
--	--

	市？”，用于制定巡演城市决策。两个场景均经访谈确认高频出现，访谈对象表示“如果免费，愿意立刻使用”，为冷启动策略提供了直接依据。
市场分析	（一）存量市场 本项目聚焦“中小音乐机构的数据问答”细分场景。 依据国家音乐产业年度报告及企业信息平台公开数据（口径为估计），我国音乐娱乐类在册企业超过 60 万家，剔除头部唱片公司与平台型企业后，可触达的中小机构约在 10 万家量级；其中已建立数据库管理作品与用户数据的比例，随数字化普及逐年上升。按每家机构每年 1200 元（约 100 元/月）的数据工具预算保守估计，仅存量市场的年规模即约 1.2 亿元。 （二）增量市场 大模型能力的普及正在创造增量：一是“自然语言查

	数据库”从技术尝鲜变为业务刚需，越来越多机构愿意为 AI 数据工具付费；二是同类需求在演出票务、在线教育、社区运营等相邻行业同样存在，产品核心能力（NL2SQL + 安全防护）具备跨行业复制潜力；三是国产大模型 API 价格持续下降（本项目使用的 GLM 系列模型即有长期免费档），使服务小微客户的毛利结构首次成立。 （三）竞争格局 竞品分两类：传统 BI（FineBI、Tableau 等）功能强大但价格与门槛双高，主要服务中大型企业；通用大模型对话产品虽能生成 SQL，但不直连数据库、无安全防护、结果不可控。本项目的空间在于两者之间的“夹缝市场”——垂直行业、轻量部署、安全可信、价格亲民。 （四）细分场景测算
--	--

按场景拆分看：独立音乐人工作室全国数万家，单客户价值低但基数大，适合免费版走量；音乐培训机构约 8 万家，客单价与付费意愿适中，是订阅版主力；厂牌与 Livehouse 数量有限但数据复杂度高，是私有化定制版的目标客群。三类场景分别对应产品的三档服务，市场结构完整。以上市场判断经小组讨论校准，并在方案书财务部分采用保守口径测算。

（三）趋势与窗口期

国产大模型 API 价格两年内下降超过一个数量级，Flash 档免费模型已能支撑本项目核心调用——这意味着“自然语言查数据库”的服务边际成本趋近于零，中小机构市场第一次具备商业可行性；同时信创与国产化替代趋势为国产开源产品提供了难得的窗口期。

	（四）竞争壁垒讨论 单纯 NL2SQL 的技术门槛正在快速降低，小组讨论认为壁垒不在算法而在三处积累：行业字段说明与提示词的调优经验、安全防线的工程细节、垂直场景的数据模型。这也是团队决定深扎音乐垂直场景、而非做通用工具的原因。
营 销 计 划 和 策 略	（一）获客策略：先开源，后订阅 冷启动阶段以“开源 + 社群”为主引擎。项目代码已在 GitHub 公开（含完整技术文档），一方面借助开源社区的传播获得第一批技术型种子用户，另一方面以“可白嫖的开源版”打消中小机构的使用顾虑；同时在校园音乐社团、独立音乐人社群进行点对点试用招募，收集真实反馈快速迭代。 （二）转化策略：免费版到付费版的价值阶梯 开源免费版满足“能查”的基本需求；托管订阅版在

	免费版之上提供云端部署、数据看板、多成员协作与优先技术支持，让客户为“省心”付费；私有化定制版面向对数据安全敏感的客户，提供上门部署与系统对接。三档之间价值阶梯清晰，客户升级路径自然。 （三）传播与留存 内容营销：在技术社区持续输出“NL2SQL 安全实践”“数据库设计”等干货文章（项目开发中的真实经验），建立专业形象；口碑机制：为推荐新客户的老客户提供订阅时长奖励；留存设计：查询日志沉淀的历史数据构成迁移成本，按月输出的“数据月报”形成使用习惯。 （四）节奏安排 第 1 年校园与社群试点 10 家，打磨产品与案例； 第 2 年通过案例复制扩大至 50 家，并启动与音乐
--	--

	教育机构的渠道合作；第 3 年探索相邻行业复制。 该策略经小组讨论形成，成员均明确各自获客任务。 （三）推广预算与 KPI 第一年推广预算控制在 1000 元以内（物料与社群运营），主要依靠开源社区与校园渠道零成本获客；设定三项关键指标：GitHub Star 数 100 以上、试点机构 10 家、周活跃查询 200 次，每月例会复盘达成情况。 （四）品牌策略 以“安全、可解释、可审计”为品牌核心词，所有传播内容强调“SQL 透明可复核”，与通用 AI 产品的“黑盒输出”形成鲜明区隔；每季度发布一次功能更新说明，向用户展示持续迭代的诚意。
已有基础	（一）产品基础：一个可运行、已开源的完整系统

	区别于纸面创意，本项目已经拥有可演示、可复现的工作系统：代码仓库已在 GitHub 公开（含 README、数据库脚本、测试数据生成器、答辩演示文稿），当前版本 v1.1，累计 20 个源文件、1400 余行代码；系统已完成端到端验证——自然语言问答、意图拦截、安全防护、失败自修复、日志留痕全部功能均实测通过，并保存有完整的运行截图与验证记录。 （二）技术基础：三项可出示证据的实证成果 其一，索引有效性实证：在 MySQL 8.4 上对三个代表性查询执行 EXPLAIN，确认全部命中设计索引，其中单曲播放统计查询命中复合索引并以覆盖索引方式执行（Using index，免回表）；其二，数据质量实证：内置测试数据生成器固定随机种子，按 50/100/500/200/5000 规模一键重建数据，并采用
--	--

	对数正态分布模拟真实播放热度长尾，保证排名类查询结果有区分度；其三，安全实证：对“删除所有播放记录”类危险请求的拦截测试通过，且拦截记录在日志中可查。 （三）过程基础：规范的开发与协作机制 项目全程使用 Git 版本管理，里程碑均有版本标签（v1.0 本地存档、v1.1 公开发布）；开发过程沉淀了四份提示词模板、一套数据库设计文档和逐页答辩讲稿；小组形成了“AI 产出、人工验收”的双人复核机制与三次正式讨论的会议记录。这些基础使项目具备从课程作品走向产品试点的现实条件。 （三）开发时间线 项目全程六周，节奏清晰：第 1 周选题讨论与需求分析，确定四步流水线架构；第 2 周数据库建模与建库脚本；第 3 周 Agent 核心流程与四份提示词
--	---

	模板；第 4 周安全防线与自修复机制，打 v1.0 标签存档；第 5 周界面、测试数据方案与公开发布（v1.1）；第 6 周答辩材料与彩排。 （四）问题解决记录 开发过程中沉淀了多份问题解决记录，每条都有原因分析与修复方案：MySQL 8 认证插件缺 cryptography 组件导致连接失败——补充依赖；均匀随机测试数据导致排名无区分度——改用对数正态分布；数据库服务静默掉线——错误日志落盘并补充启动检查清单；模型思考内容挤占正式回答——加大 max_tokens 并增加兜底提取。这些记录体现了完整的工程闭环。
团队情况	（一）组织结构与分工 团队设队长与队员各司其职，协作机制如下：队长担任项目负责人兼后端开发，负责数据库设计（6

	张表、4 个索引的建模与范式论证）、Agent 核心流程实现（意图判断、SQL 生成、执行校验、结果解释）与安全方案设计；队员担任前端交互与质量负责人，负责 Streamlit 界面实现、测试数据方案、功能测试与演示材料制作。两人共同参与需求讨论与关键决策。 （二）协作机制 采用“例会 + 版本管理 + 文档沉淀”三件套：每周一次例会同步进展并对分歧事项表决（已形成选题会、技术选型会、数据与安全设计会三次正式讨论记录）；所有代码经 Git 分支开发、合并前人工验收；关键结论沉淀为文档（README、项目状态笔记、答辩详案），避免知识只存在于个人头脑中。 （三）AI 作为“虚拟第三成员”的使用规范 团队将 AI 工具纳入协作流程并制定了明确规范：
--	---

	AI 承担初版代码生成、文案撰写与技术调研，但架构决策、安全方案、数据库建模必须由成员讨论后拍板；AI 产出必须经人工验收（运行测试、改动实验、EXPLAIN 实证）方可合入；密钥等敏感信息一律不进入代码仓库。该机制在保证效率的同时确保每位成员对系统有完整、可答辩的理解。 （三）例会与决策记录示例 技术选型会上，成员对比“直连大模型生成 SQL”与“分步流水线”两条路线，从出错定位难度、安全管控粒度、演示效果三个维度打分，一致选择后者；数据与安全设计会上，围绕“测试数据要不要使用真实姓名”展开讨论，基于隐私保护与可复现性考虑，确定 Faker 仿真数据加固定种子方案。两次会议的分歧点、论据与结论均有文字记录。 （四）质量门禁
--	--

	所有合入主干的代码必须通过三道检查：功能测试用例（7 条全部通过）、安全扫描（密钥不入仓库、公开发布前全历史密钥扫描）、文档同步（README 与项目状态笔记同步更新）。门禁未过不得合并，保证主干随时处于可演示状态。
项目特点	（一）产品层面：安全可信是最大差异点 同类自然语言查库产品大多“能生成 SQL”便止步，本项目在生成之外构建了完整的执行安全链：白名单 + 黑名单双重拦截、参数化查询防注入、被拦截请求留痕可审计。对中小机构而言，“敢把数据库交给它”比“多几个花哨功能”重要得多。 （二）技术层面：自修复与单一数据源 执行失败自动回传修正的容错闭环显著降低了误报率，用户无需理解 SQL 报错；提示词运行时读取建表脚本的设计使系统具备“表结构演进零提示词维

	护”的特性，这两点是同类课程作品与轻量产品中少见的工程细节。 （三）数据层面：仿真数据带业务灵魂 测试数据不是随机噪声：对数正态分布让播放量呈现真实的长尾形态，使”播放量 TOP-N”等演示场景具备业务说服力；固定随机种子保证任何人在任何机器上重建完全一致的数据集，可复现、可答辩。 （四）模式层面：AI + 人工验收的开发范式 团队把”如何与 AI 高效协作”本身做成了方法论：AI 负责初稿，人类负责决策与验收，所有 AI 产出经过运行测试与改动实验方可合入，并形成规范文档。这一范式既是开发效率的来源，也是团队应对 AI 时代软件工程的方式。 （五）生态层面：全程开源 从代码、文档到答辩材料全部公开（GitHub），既
--	--

	以透明换取早期用户信任，也借助社区力量持续改进。 （六）可复现性 从数据生成到测试结果全部可复现：固定随机种子、版本标签、EXPLAIN 报告留档，任何评审者都能在自己的机器上重建同样的系统与数据集，这种“可验证的诚实”在同类项目中并不多见。 （七）成本结构 核心调用使用免费档大模型，服务边际成本趋近于零，使产品可以长期向小微客户免费开放——这是同类商业产品难以跟进的成本优势，也是口碑传播的经济基础。
盈利模式	（一）挣的是什么钱 本项目挣的是“替客户省下的技术人力成本钱”：客户原本要么雇兼职数据人员（市场价每月 3000 元

	起），要么每次查询求人帮忙（时间成本高、响应慢）。我们以远低于此的价格提供“随时可问、即问即答”的数据能力，客户支付的是确定性效率的溢价。 （二）收入结构 1. 开源版：0 元，承担获客与口碑职能，不直接盈利；2. 托管订阅版：99 元/月/机构（年付 999 元），提供云端部署、看板与支持服务，是主要经常性收入；3. 私有化定制版：9800 元/次起（含部署与对接），另收年维护费（合同额 15%），面向数据敏感客户；4. 定制开发：按人天计费，用于特殊数据源接入。 （三）收费逻辑 免费版与付费版的分界线刻意设计在“多成员协作 + 云端免运维 + 技术支持”上，而非核心查询能力—
--	--

	—保证个人用户与试点客户长期免费可用，团队收入只来自机构客户的“省心溢价”，符合中小客户“先见效后付费”的心理。 （三）定价依据 99 元/月的定价锚点是“兼职数据人员月薪 3000 元的 1/30”：客户每月只需用系统替代一次人工取数即可回本；私有化版本定价参考同类轻量工具市场价的下限，主打“比定制开发便宜一个数量级”。定价经小组讨论与同类产品调研双重校准。
财务分析	（一）成本假设 1. 云服务器：入门配置 2 台，约 200 元/月（2400 元/年）；2. 大模型调用：采用 GLM 系列 Flash 档免费模型，付费场景按调用量预估不超过 100 元/月（1200 元/年）；3. 域名与杂项：约 300 元/年；4. 人力：初创期为团队成员投入，不

	计薪酬，仅计少量差旅与推广物料（1000 元/年）。合计第一年固定成本约 4900 元，成本结构极轻。 （二）收入推导（保守口径） 第 1 年：试点 10 家机构，其中 4 家转化为订阅客户（$4 \times 999 \text{ 元} \approx 4000 \text{ 元}$），另有 1 家私有化部署（9800 元），年收入约 1.38 万元，扣除成本后基本打平——第一年的目标是跑通产品与案例，而非盈利。 第 2 年：订阅客户增长至 25 家（$25 \times 999 \text{ 元} \approx 2.5 \text{ 万元}$）、私有化 2 单（约 2 万元），年收入约 4.5 万元，成本上升至约 1.2 万元（增加一台服务器与推广投入），实现微利并跨过盈亏平衡点。 第 3 年：订阅 60 家（约 6 万元）、私有化 4 单
--	---

	（约 4 万元）、相邻行业复制 1 批（约 2 万元），年收入约 12 万元，净利润约 8 万元。 （三）敏感性说明 上述测算的关键假设是“付费转化率 40%、月留存 90%”。若转化率降至 25%，第 3 年收入约 7.5 万元仍可盈利；若免费模型政策收紧导致调用成本上升，可切换至其他国产模型（系统具备 OpenAI 兼容接口，切换零代码改动），成本可控。整体而言，本项目“轻成本、慢启动、靠案例滚动”的财务路径在学生创业的约束条件下具备可行性。财务口径经小组讨论按保守原则统一。 （三）盈亏平衡分析 固定成本 F 约每年 4900 元，单客户年贡献约 999 元，边际成本趋近于零，则盈亏平衡点为 $F/999 \approx 5$ 家付费客户，预计第 2 年上半年即可达成。敏感
--	---

	性分析：若付费转化率从 40% 降至 25%，平衡点后移约半年，仍在可控范围。 （四）资金与资源需求 项目当前无外部融资需求。若后续加速扩张，预计需要 5 万元级启动资金用于服务器扩容与市场推广，缺口可通过大学生创新创业竞赛奖金与云厂商高校扶持计划覆盖，不构成启动障碍。
--	--